

HelixConstitution

HelixConstitution

모든 프로젝트가 상속하는 보편적 엔지니어링 헌장 — 기계적으로 강제되는 반-허세 법칙, 하나의 **Git** 서브모듈로 공유됨

요약

HelixConstitution는 모든 Helix/vasic-digital 프로젝트가 Git 서브모듈로 추가하는 단일한 프로젝트 중립적 규칙집으로, 협상의 여지가 없는 엔지니어링 원칙(반-허세, 증거 기반 검증, 데이터/호스트 안전성, 문서화 및 테스트 커버리지)을 코드화하여 140개 이상의 저장소로 전파합니다. 이는 전체 프로젝트군을 일관성 있게 유지하는 거버넌스 중추입니다.

간략 설명

Git 서브모듈로 제공되는 보편적이며 상속 가능한 Constitution입니다. 모든 프로젝트가 자동으로 상속하며 확장할 수는 있지만 약화시킬 수 없는 필수적이고 협상의 여지가 없는 규칙들 — 반-허세 증거 게이트, 거짓 양성 면역, 데이터 및 호스트 안전성, 커버리지 및 문서화 원칙 — 을 정의합니다.

상세 설명

HelixConstitution는 Git 서브모듈로 추가함으로써 참여하는 모든 프로젝트가 공유하는 엔지니어링 관행의 표준이자 유일한 진실의 원천입니다. 이는 코드와 마찬가지로 배포되고 버전으로 고정되는 엔지니어링 법전입니다. 핵심 문서인 Constitution.md는 약 1MB 분량의 지속적으로 버전 관리되는 문서로, 번호가 매겨진 조항들(현재 §11.4.170까지의 §11.4.x 규약군)과 이를 참조하는 에이전트별 운영 매뉴얼(CLAUDE.md, AGENTS.md, QWEN.md, GEMINI.md)로 구성되어 있어, 사람과 모든 CLI 에이전트가 동일한 규칙집을 읽게 됩니다. 상속 구조는 의도적으로 세 단계로 나뉩니다: 보편적 기반(이 서브모듈), 프로젝트 계층(프로젝트 자체의 Constitution/CLAUDE/AGENTS로 이를 확장), 그리고 선택적인 하위 디렉토리 계층입니다. 이는 위에서 아래로 평가되며, 프로젝트는 규칙을 강화할 수는 있지만 약화는 구

조적으로 금지됩니다. 그 결과 140개 이상의 저장소로 구성된 프로젝트군이 암묵적으로 분화될 수 없게 되는데, 이는 이들이 공유하는 원칙이 기억되는 것이 아니라 고정되기 때문입니다.

이 문서는 철저히 도메인 중립적입니다. 특정 공급업체, 하드웨어 SKU, 포트, 라이브러리 버전을 명시하는 내용은 모두 하위 프로젝트의 자체 Constitution로 내려가야 하며, 보편성은 결코 가정되지 않습니다. 규칙이 기반에 포함되기 전에 명시적인 네 가지 기준에 따라 입증되어야 합니다. 그 철학적 근간은 반-허세로, 상호 연결된 규약군 — §1.1 거짓 양성 면역, §11.4 최종 사용자 품질 규약, §11.4.6 추측 금지, §11.4.6.9 긍정적 증거 분류 체계 — 으로 표현됩니다. 이들의 결합된 효과는 하나의 명확한 기준선입니다: 출시 기준은 결코 “테스트 통과”가 아니라 “실제 사용자가 기능을 사용할 수 있는가”이며, 모든 성공 결과는 포착된 물리적 증거를 인용해야만 유효합니다. 동반 문서인 submodules-catalogue.md(142개 저장소)는 “이미지와 유사한 기능을 가진 저장소가 있는가?”라는 질문을 카탈로그 우선, 재구현 금지 원칙으로 전환하여 새로운 코드 한 줄을 작성하기 전에 자연스럽게 검토하게 만듭니다. 보조 스크립트는 중첩된 디렉토리 깊이에서도 서브모듈을 찾아내고 모든 커밋을 네 개의 독립된 Git 제공업체로 전파하여, 유일한 권위 있는 규칙집을 잃어버릴 수 없게 합니다.

왜 만들었는가

동일한 소유자가 작성한 여러 대형 제품 앱과 수십 개의 분리된 재사용 가능한 서브모듈들은 계속해서 같은 값비싼 규칙들을 재발견해야 했고, 같은 유형의 실패를 반복했다. 바로 테스트와 상태 보고서가 성공을 주장하면서도 최종 사용자에게는 기능이 작동하지 않는 경우(“PASS-블러프”와 “FAIL-블러프”)였다. Constitution의 각 법의학적 근거는 실제 사고를 기록한다(예: 2026년 5월 20일 D3 오디오 라우팅 PASS-블러프로, “사용 중 코덱” 필드가 비어 있음에도 검증이 통과된 경우, 또는 2026년 6월 25일 거대 버튼 UI가 토큰 동등성 테스트는 통과했지만 실제 화면은 깨진 경우). Constitution은 이러한 종류의 허위 성공을 한 번, 보편적으로 기계적으로 불가능하게 만들기 위해 존재한다. 그래야 규율이 프로젝트 간에 흐트러지거나 조용히 잊히지 않는다.

왜 혁신적인가

이는 엔지니어링 문화를 ‘따라주길 바라는 문서’에서 상속되고 버전이 관리되며 기계적으로 강제되는 법으로 전환시킨다. 스타일 가이드와 컴파일러의 차이이다. 하나의 서브모듈 업데이트로 전체 플릿의 규칙이 한 번에, 원자적으로, 추적 가능하게 업그레이드된다. 단 하나의 반블러프 조항도 신뢰가 아닌 구조적으로 모든 소비 저장소에 보장된다. 전파 게이트가 말 그대로 플릿 전체에서 해당 조항 번호를 검색하고, 짝을 이룬 변이 테스트가 게이트 자체가 블러프가 아님을 증명한다. 심지어 강제 조치 자체도 강제된다. 거버넌스는 아무도 읽지 않는 위키의 바람이 아니라 CI 작업으로 가리킬 수 있는 감사 가능하고 테스트 가능한 사실이 된다.

무엇이 혁신적인가

- 서브모듈로서의 **Constitution** — 엔지니어링 법이 코드처럼 배포되고 버전 고정되며, 의도적인 v1.0.0 스타일 태그와 프로젝트별 고정으로 모든 저장소가 정확히 어떤 법 버전에 구속되는지 안다.
- 반블러프를 일급 법의학적 원칙으로 — 모든 조항은 해당 운영자 지시의 원문과, 종종 그 조항을 유발한 실제 사고로 거슬러 올라가므로, 규칙집은 의견이 아닌 판례처럼 읽힌다.
- 규칙 자체의 메타 테스트(**\$1.1**) — 모든 게이트는 PASS→FAIL로 전환되어야 하는 변이와 짝을 이루므로 “게이트가 허위가 아니다”는 주장이 아니라 매 실행마다 증명된다. 절대 실패하지 않는 게이트는 아예 없는 게이트보다 더 나쁘게 취급된다.
- 획득된 보편성 — 명시적인 네 부분 테스트로 규칙이 진정으로 보편적인지 아니면 프로젝트 특화적인지 판단하여, 기본 규칙집을 간결하고 이식 가능하며 벤더 유출 없이 유지한다.

모든 제품에서 어떻게 사용되는가(제공하는 기능)

필수 거버넌스 기둥으로서, HelixConstitution는 가족이 참고하는 문서가 아니다. 가족이 그 위에 세워지는 하중 지지 구조다.

- 거버넌스 중추: 모든 Helix/바식-디지털 프로젝트는 이를 서브모듈로 추가하고 CLAUDE.md / AGENTS.md / QWEN.md 또는 자체 Constitution.md에서 가져온다. 규칙은 첫 커밋부터 무조건 적용되며, 프로젝트별 예외는 없다.
- 게이트와 의무: 네 단계 커버리지 모델(소스 존재, 빌드 통과, 런타임 동작, 게이트 비블러프)을 정의하며, 기능은 네 단계 모두를 통과해야 완료된 것으로 간주된다. 또한 점점 늘어나는 명명된 의무 목록이 있다: 자격 증명 처리(\$11.4.10), 항상 동기화되는 문서(\$11.4.60), 컨테이너 서브모듈 의무(\$11.4.76), CodeGraph(\$11.4.78), 필수 테스트 유형 커버리지(\$11.4.169) 등.
- 전파: CM-COVENANT-114-NNN-PROPAGATION 게이트는 소비 플릿 전체에 문자 그대로의 조항 텍스트가 존재함을 단언하므로, 조항이 부동산의 한 구석에서 조용히 사라질 수 없다. 미준수는 플래그로 회피할 수 없는 치명적인 릴리스 블로커다.
- 발견: submodules-catalogue.md는 “X를 수행하는 모듈을 이미 보유하고 있는가?”라는 질문에 한눈에 답할 수 있게 하여, 중복 노력을 원천 차단한다.
- AI 에이전트의 일관성: 동일한 법이 모든 CLI 에이전트(Claude Code, Codex/Cursor/Aider/OpenCode/Crush/Kimi via AGENTS.md, Qwen Code via QWEN.md)에 동일하게 표현되므로, 어떤 도구가 코드를 건드리든 동일한 규약을 따른다.

가장 큰 기술적 도전 과제와 해결 방법

- 임의 깊이로 중첩된 서브모듈에서 규칙 찾기 — 세 단계 깊이의 서브모듈에 묻힌 규칙이라도 자신이 어디에 있는지 모르더라도 법을 찾아야 함
→ find_constitution.sh는 상위 디렉터리를 거슬러 올라가며 깃 슈퍼

프로젝트 포인터를 재귀적으로 따라가고, CONSTITUTION_DIR 오버라이드와 두 가지 지원 레이아웃(constitution/, submodules/constitution/)을 준수하여 중첩 깊이에 관계없이 결정론적으로 위치를 파악함.

- 네 개의 깃 제공자 간 단일 저장소 권한 유지 — 동기화가 어긋난 미러는 무용지물 → `install_upstreams.sh`는 선언형 `Upstreams/*.sh` 원격 저장소를 읽어 origin에 여러 푸시 URL을 설정하여, 단일 `git push` 명령으로 GitHub(주 저장소), GitLab, GitFlic, GitVerse로 원자적으로 팬아웃되며 어떤 미러도 뒤쳐지지 않도록 함.
- 범용 기반으로의 규칙 팽창/프로젝트 유출 방지 — “그냥 여기에 추가하자”라는 유혹이 반복되면 이식성이 점점 떨어짐 → 획득된 범용성의 네 가지 기준과 §11.4.17 범용-프로젝트 분류 원칙을 모든 새 규칙에 적용하여, 프로젝트별 요구사항은 반드시 프로젝트 계층으로 되돌려 보냄.
- 상속 게이트의 실제 작동 검증 — 실패하는 모습을 한 번도 보지 못한 게이트는 신뢰할 수 없는 게이트 → `meta_test_inheritance.sh`라는 센티넬 메타 테스트는 §11.4 앵커를 의도적으로 삭제하고 게이트가 이를 감지하는지 확인하여, 강제 메커니즘 자체가 침묵의 오류에 대해 지속적으로 재검증되도록 함.

기술 스택

- 깃 서브모듈 상속 — 이유: 깃 서브모듈은 규칙집을 권위 있게 유지하면서 소버라이더로 버전을 고정할 수 있는 유일한 메커니즘이며, 복사-붙여넣기의 침묵 대신 명시적이고 검토 가능한 버전 업그레이드를 가능하게 함. 방법: 소비 프로젝트는 서브모듈을 추가하고 에이전트 파일을 @import하며, 세 계층은 상위에서 하위로 엄격한 ‘확장-약화 금지’ 계약을 유지하며 평가됨.
- `find_constitution.sh` — 이유: 규칙이 아무리 깊이 중첩되어도 안정적으로 찾을 수 없다면 무용지물이며, 경로를 하드코딩하면 프로젝트 재구성 시 즉시 깨짐. 방법: 상위 디렉터리 탐색과 `git rev-parse --show-superproject-working-tree` 재귀 호출을 통해 두 가지 지원 레이아웃을 해결하며, CONSTITUTION_DIR 오버라이드로 보완됨.
- `install_upstreams.sh + Upstreams/` — 이유: 네 제공자 간 중복성은 유지 노력이 제로일 때만 실효성이 있으며, 그렇지 않으면 미러는 곧 퇴화함. 방법: 선언형 원격 저장소별 .sh 파일을 단일 멀티-URL origin으로 구체화하여 네 번의 푸시를 하나로 통합함.
- §1.1 변형 메타 테스트 — 이유: 절대 실패하지 않는 게이트는 아예 없는 것보다 나쁨. 거짓된 신뢰를 심어주기 때문. 방법: 각 게이트는 `sed` 삭제/이름 변경 변형을 반드시 `PASS→FAIL`로 전환한 후 복원하여, 모든 게이트가 매 실행마다 여전히 작동함을 증명함.
- 전파 게이트(**CM-COVENANT-114-NNN-PROPAGATION**) — 이유: 규약이 주력 저장소뿐만 아니라 모든 소비자에게 검증 가능하게 존재할 때만 범용적임. 방법: 소비자 전체에 걸친 리터럴 조항 번호 `grep`과, 전파 검사 자체가 실패할 수 있음을 증명하는 §1.1 변형 메타 테스트로 뒷받침됨.
- `submodules-catalogue.md(§11.4.74)` — 이유: 중복 방지 원칙을 가장 빠르게 위반하는 방법은 이미 소유한 것을 모른 채 새로운 것을 만드는 것. 방법: 142개 저장소의 기능별 그룹화 목록으로, 새로운 항목이 스캐폴딩되기 전에 카탈로그 검사가 트래커에 기록됨.

- 다중 형식 내보내기 — 이유: 동일한 법은 인간이 읽고, 도구가 파싱하고, 아카이브가 보존하는 데 동등하게 활용 가능해야 함. 방법: 모든 표준 문서는 하나의 소스에서 .md/.html/.pdf/.docx 형식으로 내보내짐.

내용

상태 및 정직성 고지

- 상태: 배포 완료. 현재 버전 관리 중이며, 전 제품군(공식 및 미러 저장소)에 걸쳐 하위 모듈로 활성 사용 중.
- 라이선스: 미정 — 검토한 소스 자료에 명시되지 않음. 게시 전 저장소의 LICENSE 파일과 반드시 확인 요망.
- 추가 업스트림 미러: GitLab `helixdevelopment1/helixconstitution`, GitFlic `helixdevelopment/helixconstitution`, GitVerse `helixdevelopment/HelixConstitution`.

우선순위 등급: Helix-주축 — Helix 제품군의 모든 구성 요소를 구축하는 필수 거버넌스 기둥.